DAC Testnet

DAC SENDER — Testnet Transaction Interface

Prepared by	JERUZZALEM – DAC Infra Tester
Date	Friday, 22 May 2026
Environment	Windows (Node :8545) + WSL Ubuntu (Node :8546) — Indonesia / CGNAT
Report Type	Application Development Report — On-Chain Tooling & Infrastructure Testing

Executive Summary

This report documents the design, development, and deployment of DAC•SENDER (v1.4.3) — a client-side web interface built to support infrastructure testing of the DAC Quantum Chain Testnet. The tool was developed to generate diverse, measurable transaction volume across multiple EVM execution types: native transfers, contract calls, contract creation, internal transactions, and NFT minting. All findings in this report are based on direct observation from a dual-node, single-machine environment operating under CGNAT residential network constraints in Indonesia.

Finding	Status
DAC•SENDER v1.4.3 deployed and operational	Confirmed
Gas price anomaly — 2,824 Gwei spike observed	Reported to DAC team
Protocol fee contract — percentage cap bug identified and patched	Resolved
NFT Launchpad with on-chain registry fully operational	Confirmed
RPC 502/504 errors — handled gracefully in frontend	Mitigated
EVM London compatibility — PUSH0 opcode absent	Confirmed

Tool Overview

DAC•SENDER is a static, no-build web interface hosted on GitHub Pages. It requires no backend server, no build pipeline, and no external API beyond the DAC Testnet RPC endpoint and Pinata IPFS. The tool is designed for direct browser deployment and operates entirely client-side using ethers.js v6.

Live Endpoints

- Main Interface: https://EdLWEISS186.github.io/dac-dual-node-cgnat-setup/Sender-Web/
- NFT Launchpad: https://EdLWEISS186.github.io/dac-dual-node-cgnat-setup/Sender-Web/mint.html

Feature	Description
Send DACC	Native token transfer — single send and multi-send (batch) to up to 50 recipients in one transaction via DACMultiSend contract
Deploy Contract	One-click ERC-compatible contract deployment (DACInception) from the browser
Deploy NFT	Full ERC-721 collection deployment with IPFS artwork upload via Pinata and automatic registry registration
Protocol Fee	Optional fee routing through DACSendProxy — 5% of gas cost, capped at 1 DACC absolute maximum
NFT Launchpad	Public mint portal reading all deployed collections from on-chain DACNFTRegistry
Real-time Stats	Live TPS, block time, RPC latency, gas price — updated every 13 seconds
Metrics Export	CSV export of full session transaction history
Bridge	UI placeholder — pending DAC team bridge contract and relayer infrastructure

Testing Environment

Parameter	Value
Network	DAC Quantum Chain Testnet
Chain ID	21894 (0x5586)
RPC (public)	https://rpctest.dachain.tech
RPC (local — Windows)	http://127.0.0.1:8545
RPC (local — WSL)	http://127.0.0.1:8546
Explorer	https://exptest.dachain.tech
EVM Version	London (no PUSH0 opcode)
Wallet (primary)	0x870ad63acc507cdfd878F170606d19ae78988AFE
Wallet (secondary)	0x2a01EAA3175a2796acD8F64fCe6dD6488D39D3Af
ethers.js	v6.7.0 (CDN — no build step)
IPFS Provider	Pinata — scoped API key (pinFileToIPFS + pinJSONToIPFS only)

Deployed Smart Contracts

All contracts were compiled with solc 0.8.14, evmVersion: london, optimizer enabled (200 runs). Deployment was performed directly from the browser via ethers.js ContractFactory.

Contract	Address	Purpose
DACNFTRegistry	0x34CDf37FeEb81877acabAD601AAaA3AE5a5029Ae	Shared on-chain registry of all NFT collections deployed through DAC•SENDER
DACSendProxy	Deployed per-user (localStorage)	Protocol fee routing — 5% of gas cost, 1 DACC absolute cap
DACMultiSend	Deployed per-user (localStorage)	Batch send to up to 50 recipients in a single transaction
DACInception	Deployed per session	ERC-compatible contract — interaction counter, deploy proof
DACInceptionNFT	Deployed per collection	ERC-721 — single-image collection with public mint, supply limits, per-wallet cap

Contract Architecture

The architecture follows a hub-and-spoke model. DACNFTRegistry is the single shared on-chain state store, deployed once by the tooling maintainer. All other contracts are user-deployed on demand. The frontend auto-deploys proxy and multi-send contracts on first use, storing addresses in localStorage for session persistence.

Technical Findings

Finding 1 — Gas Price Anomaly (2,824 Gwei Spike)

During active stress testing sessions, the DAC Testnet gas price exhibited extreme and unexplained volatility, spiking from a baseline of approximately 0.826 Gwei to 2,824 Gwei — a 3,400× increase — without a corresponding increase in network utilization.

Date	Gas Price	Avg Fee (Coin Transfer)
May 2026 (baseline)	~0.826 Gwei	~0.00002 DACC
May 2026 (anomaly)	2,824.2 Gwei	~0.041 DACC
May 2026 (peak)	>3,000 Gwei	~3 DACC (contract call)

The anomaly directly impacted the protocol fee calculation in DACSendProxy, which computes fees as a function of tx.gasprice. At 2,824 Gwei, even simple native transfers became expensive, and the fee cap mechanism triggered unexpectedly. This finding was formally reported to the DAC team via the official community channel.

Finding 2 — Protocol Fee Contract — Percentage Cap Bug

The initial deployment of DACSendProxy implemented a percentage-based fee cap:

```
uint256 fee = tx.gasprice * 21000 * 5 / 100;    // intended: 5% of gas fee
uint256 cap = msg.value / 50;                    // BUG: 2% of sent amount
```

During a 150 DACC test transfer at elevated gas prices, the cap triggered at msg.value / 50 = 3 DACC — a fee disproportionate to the intended design. The root cause was that the percentage-of-amount cap was never intended to be the active mechanism; it existed as a safety fallback, but became the operative fee at high gas prices.

Resolution: The cap was replaced with a fixed absolute maximum of 1 DACC. This ensures the protocol fee remains invisible at normal gas prices (~0.000001 DACC) while providing a hard ceiling during anomalous conditions. Redeployment required clearing the localStorage proxy address key (dac_send_proxy_addr) to force fresh contract instantiation.

Finding 3 — EVM London Compatibility — No PUSH0 Opcode

DAC Testnet is based on Geth v1.10.5, which implements the London EVM hardfork. The PUSH0 opcode (EIP-3855, introduced in Shanghai) is not available. All Solidity contracts were compiled with evmVersion: london to prevent the compiler from emitting PUSH0, which would cause deployment failures on the DAC EVM.

This constraint was identified early in development and applied consistently across all five smart contracts: DACNFTRegistry, DACSendProxy, DACMultiSend, DACInception, and DACInceptionNFT.

Finding 4 — RPC Availability and 502/504 Errors

The public RPC endpoint (rpctest.dachain.tech) experienced recurring 502 Bad Gateway and 504 Gateway Timeout errors throughout the development and testing period. These are consistent with

findings documented in the RPC Availability report (05 May 2026) and Infrastructure Validation report (07 May 2026).

Impact on DAC•SENDER:

- NFT collection loading in mint.html would fail silently, showing a blank state
- My Collections tab would show generic error text without actionable guidance
- Stats bar would display stale or missing network metrics

Mitigation: Error handling was implemented throughout the application to distinguish RPC availability failures (502/504/SERVER_ERROR) from application logic errors. Affected UI panels display a contextual RPC Unavailable message with a Retry button. The stats bar transitions to an OFFLINE state with visual indicator.

Finding 5 — NFT Launchpad — On-Chain Registry Architecture

The initial design stored deployed NFT collection metadata in browser localStorage — a per-device, non-shared store. This architecture was insufficient for a community launchpad where collections deployed by one user must be discoverable by all users.

The solution implemented is a shared on-chain registry contract (DACNFTRegistry) deployed once at a known canonical address. Every NFT collection deployed through DAC•SENDER automatically calls DACNFTRegistry.register() after successful deployment, recording: contract address, name, symbol, max supply, mint price, image CID, and metadata CID. The mint.html launchpad reads from this registry via DACNFTRegistry.getAll(), providing a fully decentralized, backend-free shared state.

This eliminates the need for a centralized database, API server, or manual curation. Any NFT collection deployed through DAC•SENDER is automatically and permanently visible to all community users on the launchpad.

Transaction Volume Generated

DAC•SENDER was specifically designed to produce transaction patterns representative of a real multi-feature dApp, rather than repetitive single-type transfers. The following execution types are generated:

Transaction Type	Mechanism	Gas Consumption
Native token transfer	Direct EOA-to-EOA (ethers.js sendTransaction)	~21,000 gas
Proxy transfer	DACSendProxy.sendTo() — internal transfer + fee split	~60,000 gas
Batch transfer	DACMultiSend.multiSend() — up to 50 recipients	~200,000–450,000 gas
Contract deployment	DACInception via ethers.js ContractFactory	~380,000–420,000 gas
NFT deployment	DACInceptionNFT — ERC-721 with supply and mint controls	~1,200,000–1,400,000 gas
Registry registration	DACNFTRegistry.register() — post-NFT-deploy call	~120,000–160,000 gas
NFT mint	DACInceptionNFT.mint(quantity) — payable	~70,000–130,000 gas

The combination of these transaction types produces diverse chain activity spanning the full spectrum of EVM execution complexity — from simple value transfers to multi-step contract interactions with internal transactions, event emissions, and cross-contract calls.

JavaScript Source Obfuscation

The JavaScript source of index.html is fully obfuscated prior to deployment. Obfuscation is applied to the entire client-side script block using javascript-obfuscator with RC4 string encoding, control flow flattening, and dead code injection. The obfuscated output replaces the source script block in its entirety.

The primary purpose is to deter unauthorized extraction of embedded sensitive constants:

- Pinata JWT (IPFS upload credential — domain-restricted to edlweiss186.github.io)
- DEV_WALLET address (protocol fee recipient)
- Contract bytecodes for all five deployed contracts
- 93-address random recipient pool for stress testing

HTML markup and CSS remain unobfuscated. The obfuscation applies exclusively to the JavaScript execution layer. This approach is a deterrent, not a cryptographic security control — it raises the cost of credential extraction to a level disproportionate to the value of testnet credentials.

IPFS Integration — Pinata

NFT artwork and metadata are uploaded to IPFS via Pinata's pinning service. The API key used for this integration was provisioned with minimal permissions — specifically pinFileToIPFS and pinJSONToIPFS under the Legacy Endpoints scope only. All V3 resource permissions are set to None.

Parameter	Value
Provider	Pinata (pinata.cloud)
Key Name	DAC-NFT[DAC•SENDER]
Permissions	pinFileToIPFS + pinJSONToIPFS only
Domain Restriction	https://edlweiss186.github.io
Replication	FRA1 + NYC1 (Pinata default)
Image Format	ipfs://{imageCID} — stored in metadata JSON
Metadata Format	ERC-721 compatible: name, description, image

Version History

Version	Date	Key Changes
v1.0.0	May 2026	Initial deployment — Send DACC (single send), wallet connect overlay, basic transaction log
v1.1.0	May 2026	Quick-fill buttons (0.001, 0.01, 0.1, MAX), recipient address field, balance display
v1.2.0	May 2026	Deploy Contract tab (DACInception), contract interaction counter, deploy log panel
v1.3.0	May 2026	Multi-Send tab (DACMultiSend), batch dispatch, Randomize All, Metrics Export CSV
v1.3.1	May 2026	CSS alignment hotfix — Multi-Send address row spacing
v1.4.0	May 2026	Deploy NFT tab, Pinata IPFS integration, DACNFTRegistry, NFT Launchpad (mint.html), mint link generation
v1.4.1	May 2026	Protocol fee fix — 1 DACC absolute cap replaces percentage-based cap
v1.4.2	May 2026	Bridge tab (Coming Soon), connect wallet overlay on mint.html, topbar subtitle
v1.4.3	May 2026	Real-time stats bar (TPS, block time, RPC latency, gas), mint.html 2-tab layout, transaction log panel height fix, split IPFS links

Recommendations

For DAC Team

- Stabilize and expand public RPC backend infrastructure — the single-endpoint architecture at rpctest.dachain.tech continues to exhibit gateway timeout behavior under load, directly impacting dApp usability.
- Publish official bridge contract addresses and relayer infrastructure to enable the Bridge feature — the frontend integration is complete pending these dependencies.
- Confirm gas pricing mechanism and expected baseline — the 2,824 Gwei anomaly had material impact on protocol fee behavior and stress testing cost predictability.
- Audit and complete EIP-1559 (eth_feeHistory) implementation — MetaMask compatibility issues documented in the 09 May 2026 report remain unresolved and affect community onboarding.

For Testnet Participants

- Use Rabby Wallet or OKX Wallet as the primary wallet client — MetaMask EIP-1559 compatibility issues documented separately continue to cause transaction failures.
- Self-host a local DAC node and connect the wallet to localhost RPC for improved transaction reliability, particularly during public RPC outages.
- Verify mint status on the block explorer before retrying failed transactions — the stale UI desynchronization issue documented in Report 4 (10 May 2026) may cause misleading frontend states.

Conclusion

DAC•SENDER v1.4.3 is a fully operational, no-build, no-backend transaction interface for the DAC Quantum Chain Testnet. It provides a structured mechanism for generating diverse EVM transaction types — from simple native transfers to full NFT collection deployments — under controlled, measurable conditions. The NFT Launchpad component extends the tool into community-facing territory, enabling any wallet to discover and mint collections deployed through the interface.

The technical findings documented during development contribute directly to the broader infrastructure investigation series initiated in May 2026. The gas price anomaly, EVM compatibility constraints, RPC availability issues, and protocol fee contract behavior are all directly observable through this tooling and provide measurable, reproducible data points for DAC team reference.

Layer / Component	Status
DAC•SENDER v1.4.3	Deployed and operational — GitHub Pages
Smart contracts (5)	Compiled and deployed on DAC Testnet
DACNFTRegistry	0x34CDf37FeEb81877acabAD601AAaA3AE5a5029Ae — active
Protocol fee (DACSendProxy)	Patched — 1 DACC absolute cap, v1.4.1+
NFT Launchpad (mint.html)	Operational — on-chain registry integration confirmed
Gas price anomaly	Reported to DAC team — monitoring ongoing
RPC error handling	Implemented — graceful degradation with Retry UI
Bridge feature	Pending DAC team bridge infrastructure